📋 Relatório Diário de Desenvolvimento

Projeto: Extrator de Dados ESL Cloud (ETL) **Data:** 18/11/2025 **Status:** ☑ Concluído com Sucesso

---

🎯 Resumo das Entregas

Hoje finalizamos com sucesso a implementação completa da extração de dois relatórios financeiros críticos utilizando a API **Data Export** (Método `GET`). Todas as classes necessárias foram criadas, a lógica de negócio foi implementada e a validação cruzada confirmou **100% de fidelidade** com os dados da planilha de referência.

🚀 Módulos Implementados e Finalizados

1. **Contas a Pagar** (Template ID: 8636)
2. **Faturas por Cliente** (Template ID: 4924)

🛠 Detalhamento das Atividades

1. Análise e Engenharia Reversa das Requisições

Iniciamos identificando as requisições exatas utilizadas no navegador para gerar os relatórios oficiais.

- Identificamos os **Template IDs** corretos.
- Mapeamos os parâmetros de filtro via método `GET`.
- Obtivemos o JSON de resposta cru e realizamos o "De/Para" com as colunas do Excel utilizado pelo financeiro.

2. Desenvolvimento e Criação de Classes

Para ambos os módulos, o ciclo de desenvolvimento backend (Java) foi concluído:

- **DTOs (Data Transfer Objects):** Criados espelhando exatamente a estrutura do JSON da API.
- **Entities:** Criadas para persistência no SQL Server, com tipagem correta (`DECIMAL` para valores, `DATETIMEOFFSET` para datas).
- **Mappers:** Implementados com regras de negócio robustas:
  - **Correção Monetária:** Tratamento de `Locale.US` para garantir que valores como `179.0` sejam lidos corretamente como `179.00` (evitando o erro de multiplicação por 10).
  - **Tradução de Domínio:** Conversão de termos técnicos (ex: `variable_costs`) para termos de negócio (ex: "4. Custos Variáveis").
  - **Tratamento de Listas:** Conversão de Arrays JSON (ex: listas de notas fiscais) para Strings separadas por vírgula.

3. Soluções de Integridade de Dados

Durante o desenvolvimento, resolvemos desafios específicos de cada relatório:

- **Em Faturas por Cliente:** Implementamos uma lógica de **Chave Única Híbrida**. Como o relatório mistura CT-es e NFS-es, o sistema identifica automaticamente qual ID utilizar (Chave do CT-e ou Número da NFS-e) para evitar duplicidade ou perda de dados.

📊 Validação e Comparação (Quality Assurance)

Realizamos a extração real via script e comparamos o CSV gerado contra a planilha de referência (Excel).

Resultado da Comparação:

Módulo	Status	Observações da Validação
Contas a Pagar	☑ **100% Validado**	Valores monetários bateram exatamente (centavos corrigidos). Status de pagamento atualizado em tempo real.
Faturas por Cliente	☑ **100% Validado**	Todos os campos capturados. Dados de CT-e e NFS-e foram salvos corretamente na mesma tabela sem conflitos.

💾 Artefatos Técnicos Entregues

As seguintes estruturas foram criadas e estão operacionais no projeto:

📦 Pacote: `modelo.dataexport.contasapagar`

- `ContasAPagarDTO.java`
- `ContasAPagarMapper.java`
- `ContasAPagarDataExportEntity.java`
- **Tabela SQL:** `faturas_a_pagar_data_export`

📦 Pacote: `modelo.dataexport.faturaporcliente`

- `FaturaPorClienteDTO.java`
- `FaturaPorClienteMapper.java`
- `FaturaPorClienteEntity.java`
- **Tabela SQL:** `faturas_por_cliente_data_export`

☑ Conclusão

O sistema está extraindo, transformando e salvando os dados corretamente via API Data Export. A substituição dos métodos antigos está pronta para ser efetivada em produção.